Dinamikus programozás: A feladatot felosztja részfeladatokra, és a részfeladat eredményét egy táblázatban tárolja el, és ezáltal elkerüli az ismételt számítást, ha a részfeladat megint felmerül.

Lépései:

1. részproblémákra bontás, optimális megoldás jellemzése
2. a részproblémák közötti összefüggések meghatározása (rekurzív képlettel)
3. a részproblémák optimális értékének kiszámítása alulról felfelé haladva (táblázat kitöltése)
4. az optimális megoldás megalkotása

(Például: n-edik Fibonacci szám kiszámolása: rekurzívan sokáig tarthat, de ha tároljuk a részeredményeket tömbbe, és rekurzió helyett egy for ciklussal írjuk meg a tömb tárolt részeredményeit felhasználva, sokkal gyorsabb.)

Alap mátrixszorzás:

```
Mátrixszorzás(A,B)
IF oszlop(A)<>sor(B) THEN ERROR
FOR i:=1 to sor(A)
  FOR j:=1 to oszlop(B)
    C(i,j):=0
    FOR k:=1 to oszlop(A)
      C(i,j) := C(i,j) + A[i,k]*B[k,j]
```

Dinamikus programozási példa: Mátrixsorozat összeszorzása

• Adott n mátrix (A1, A2, ..., An) és ezek A1*....*A2*An szorzatát akarjuk kiszámítani.

• Mátrixok szokásos szorzása + zárójelezés

• Mátrixok egy szorzata teljesen zárójelezett, ha vagy egyetlen mátrix, vagy két zárójelbe rakott teljesen zárójelezett mátrixszorzat szorzata.

• Asszociatív, bármilyen zárójelezés ugyanazt az eredményt adja,

• viszont a zárójelezés alapvetően befolyásolja a kifejezés kiértékelésének költségét.

3 mátrix: 10x100, 100x5, 5x50 => ((AB)C)-nél 10*100*5 + 10*5*50 = 7500 szorzás,
(A(BC))-nél viszont 100*5*50 + 10*100*50 = 75000 szorzás

• Szorzótényezők sor-oszlop kompatibilisek kell, hogy legyenek

• Számítási idő: Egy pxq és egy qxr dimenziós mátrix összeszorzásának a költsége p*q*r

A mátrixszorzás probléma feladata egy adott szorzás optimális zárójelezésének megtalálása. Az input:
A1 , A2 , . . . An , ahol Ai mérete $p_{i-1} \times p_i$.

Részprobléma: Ai . . . A j optimális zárójelezése minden i, j párra, a megoldás értéke legyen m[i, j]. Nyilvánvaló, hogy m[i, i] = 0.

Rekurzív összefüggés: Ha a szorzásnál az első zárójelpár hátsó zárójele az Ak után kerül, akkor a költség: m[i, k] + m[k + 1, j] + $p_{i-1}\, p_k\, p_j$. Ezen lehetőségek közül választjuk a legjobbat, így ha i < j, akkor m[i, j] = min $_{i \le k < j}$ {m[i, k] + m[k + 1, j] + $p_{i-1}\, p_k\, p_j$ }. Táblázatkitöltés: m[i,j]-hez használjuk az m[i,k] és m[k+1,j] értékeket, ezeknek kell meglenni az m[i,j] érték számításánál. Így a helyes kitöltési sorrend átlónként megy (elsőként a j=i, értékek, aztán j=i+1, majd j=i+2 és így tovább).

A megoldás meghatározását feljegyzéses módszerrel oldjuk meg, S[i,j]-ben feljegyezzük, hogy mi volt az optimális döntés m[i,j] számításakor.

Példa:

A1 (6 × 7), A2 (7 × 3), A3 (3 × 1), A4 (1 × 2), A5 (2 × 4)

Az m[i,j] értékek táblázata:

0	0	0	0	0
0	0	0	0	8
0	0	0	6	20
0	0	21	35	57
0	126	63	75	95

m[1, 3] = min{[m[1, 1] + m[2, 3] + 6 · 7 · 1, m[1, 2] + m[3, 3] + 6 · 3 · 1} = 63
m[2, 4] = min{[m[2, 2] + m[3, 4] + 7 · 3 · 2, m[2, 3] + m[4, 4] + 7 · 1 · 2} = 35
m[3, 5] = min{[m[3, 3] + m[4, 5] + 3 · 1 · 4, m[3, 4] + m[5, 5] + 3 · 2 · 4} = 20
m[1, 4] = min{[m[1, 1] + m[2, 4] + 6 · 7 · 2, m[1, 2] + m[3, 4] + 6 · 3 · 2, m[1, 3] +
m[4, 4] + 6 · 1 · 2} = 75

Az S[i,j] értékek táblázata:

0	0	0	0	0
0	0	0	0	4
0	0	0	3	3
0	0	2	3	3
0	1	1	3	3

Megoldás: ((A1)(A2 A3))(A4 A5)

Aritmetikai művelet: olyan művelet, melyet számokon végzünk és eredménye szám. Három típus:
- unáris műveletek: egyoperandusú művelet, pl. kettes komplemensképzés
- multiplikatív m.: kétoperandusú művelet, pl. szorzás, osztás, egészrészű osztás, balra/jobbra shiftelés
- additív m.: szintén kétoperandusúak, pl. összeadás, kivonás

Precedenciájuk: 1.unáris, 2.multiplikatív, 3.additív. Azonos szintűek: balról jobbra. Zárójelezéssel módosítható.

Relációs művelet: eredménye egy logikai érték (igaz, vagy hamis). A fontosabb relációs műveletek: < kisebb mint, > nagyobb mint, = == egyenlő, <> != nem egyenlő, <= kisebb vagy egyenlő, >= nagyobb vagy egyenlő. (Zárójelezhető!)
Logikai művelet: az ítéletkalkulus ítéletein definiált művelet. A műveletek eredményeinek logikai értéke csak komponenseinek logikai értékétől függ.
- negáció (¬, NOT): egyváltozós l.m., egyetlen be-/kimenet, igazra hamis és fordítva.

Kétváltozós műveletek:
- konjunkció (^, AND): csak akkor igaz ha mindkét komponens igaz.
- diszjunkció (v, OR): „megengedő vagy”, igaz ha legalább az egyik komponens igaz.
- kizáró vagy (XOR): igaz ha csak az egyik komponens igaz.
- implikáció (→): csak akkor hamis ha A igaz és B hamis, egyébként igaz.
- ekvivalencia (↔): akkor igaz, ha a két komponens logikai értéke azonosaz igaz vagy hamis.

Precedenciájuk: ¬, ^,v,→,↔

Tömb: azonos típusú elemekből álló, sorosan tárolt adatcsoport. Amennyiben hossza előre ismert, statikus tömbnek nevezik, ha hossza a program futása során (runtime) változik, dinamikus tömbnek, vagy listának nevezzük. Az elemeket indexeléssel lehet elérni, a hivatkozás a tömb nevével és az elemindexszel történik. Az indexeknek általában van alsó és felső határa, az alul- vagy túlcímzés hibát okoz.
Többdimenziós tömbök: akár n-dimenzióig bővíthetőek, minden dimenzióhoz tartozik egy index, egy elemre az egyes dimenziók indexeiből adódó pozíció meghatározásával lehet.

Karakter típusú változó: használata esetén egyetlen karaktert tudunk eltárolni, jellemzően a karakter kódját (ASCII, UTF-8 stb.) használva. ASCII kódok használatánál a karakter 1 bájton tárolódik.
Karakterlánc : mindig összetett típus (mivel tovább bontható karakterekre), valamilyen karakteres típusból áll össze. Elemei bármilyen karakterek lehetnek, hossza lehet meghatározott, vagy akár dinamikus hosszúságú is. Egy elemére a tömbökhöz hasonlóan lehet hivatkozni (mivel egy karakterlánc felfogható karakter típusú elemek egydimenziós tömbjeként), azaz a karakterlánc azonosítójával, és a karakter indexével. Egy karakterlánc helyigénye: maximális elemeinek száma, szorozva az egy karakter helyigényével, plusz még egy byte (hossznak / záró karakternek).

A videokártyáknak két alapvető üzemmódja van, az alfanumerikus és a grafikus.
Alfanumerikus üzemmód: a képernyőn csak olyan betűk, jelek jeleníthetők meg, amelyeket a karakterek pontjait tároló karaktergenerátor tartalmaz. Ekkor a megjeleníthető sorok és az egy sorba írható betűk számával jellemzik az üzemmódot, pl. 80x25-ös.
Grafikus üzemmód: a képernyő minden képpontja külön kezelhető. Ebben a módban a vízszintesen és függőlegesen megjeleníthető pontok számával adják meg a felbontást, pl. 1920x1080. Az alfanumerikus mód kisebb memóriát igényel, de ma már korlátai miatt alig használják.

Alfanumerikus képernyő kezelése: 2 bájtos rekeszekből áll, 1.bájt: szín, 2.: karakter kódja. Egy karakter írása egy megadott helyre úgy történik, hogy adott a karakter színe, kódja, valamint egy sor és egy oszlop.
Egy 80x25-ös képernyő karakterének címe: képernyő szegmenscíme + (sor ∗ 160) + oszlop ∗ 2
Grafikus képernyő kezelése pixelenként történik. A képpontok színe vagy RGB(A) formátumban van tárolva, vagy pedig egy indexet jelöl egy előre eltárolt palettában. 24 bites módban minden színösszetevőt 8 biten ábrázolunk, 32 biten van még egy 8 bites alpha érték is.